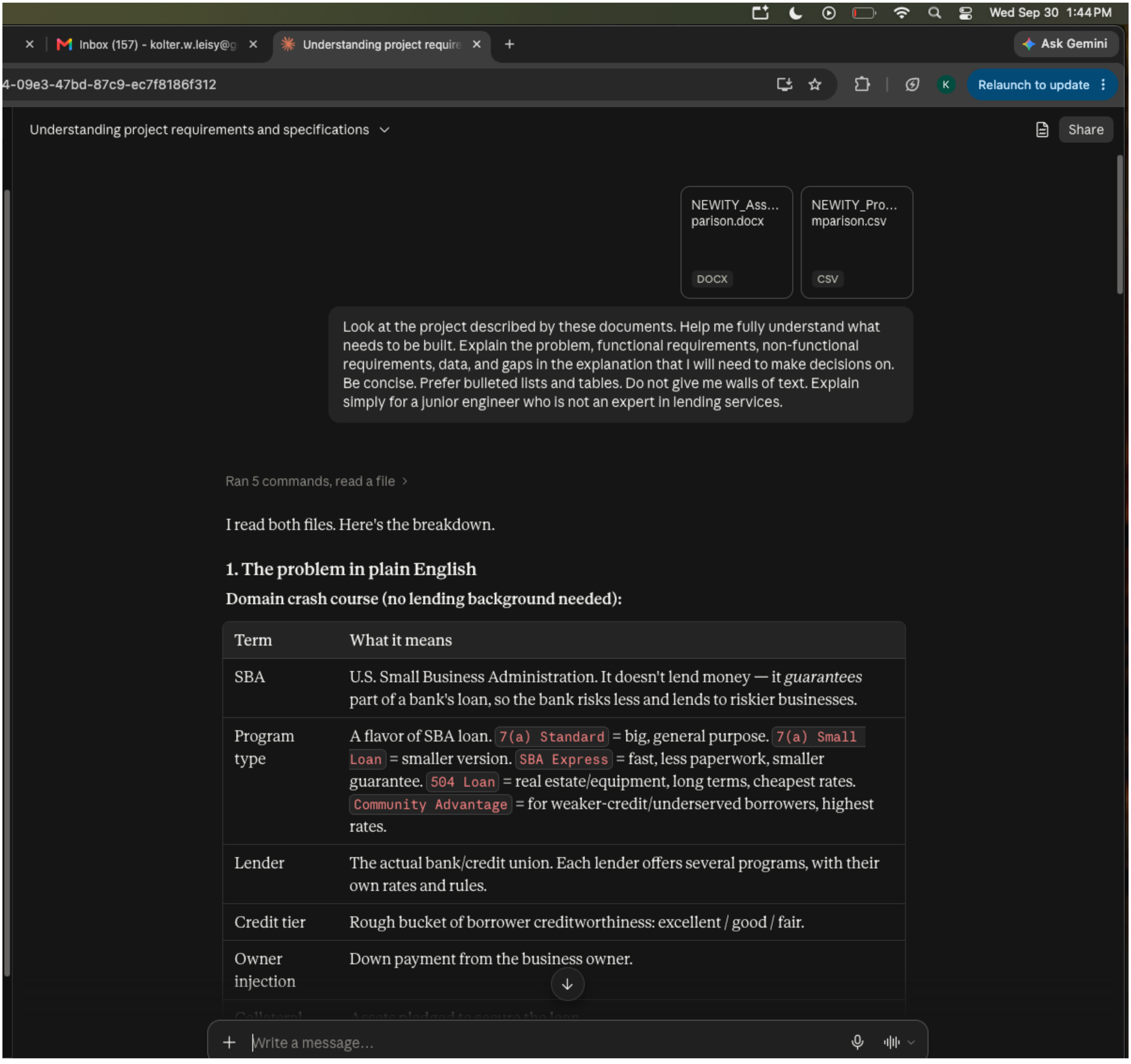
00 understanding first prompt



Requested by the team (pick a subset)

Feature	Effort	Value to a rep on a call
Filter by borrower criteria	Low	Highest — this is the product
Rate & term details	Low	High
Eligibility requirements at a glance	Low	High — reps must state conditions
Side-by-side comparison of 2–3 programs	Medium	High
Save/share a comparison with the borrower	High (needs persistence/export)	Medium — happens after the call
Flag when lender rates are updated	Medium (needs "last seen" state)	Low during a call
Manager view: most-recommended programs	High (needs logging + storage)	Low for V1, only 2 users

A strong V1 = the core + comparison + visible eligibility notes. Cut sharing, manager analytics, and staleness flagging, and explain why in the demo.

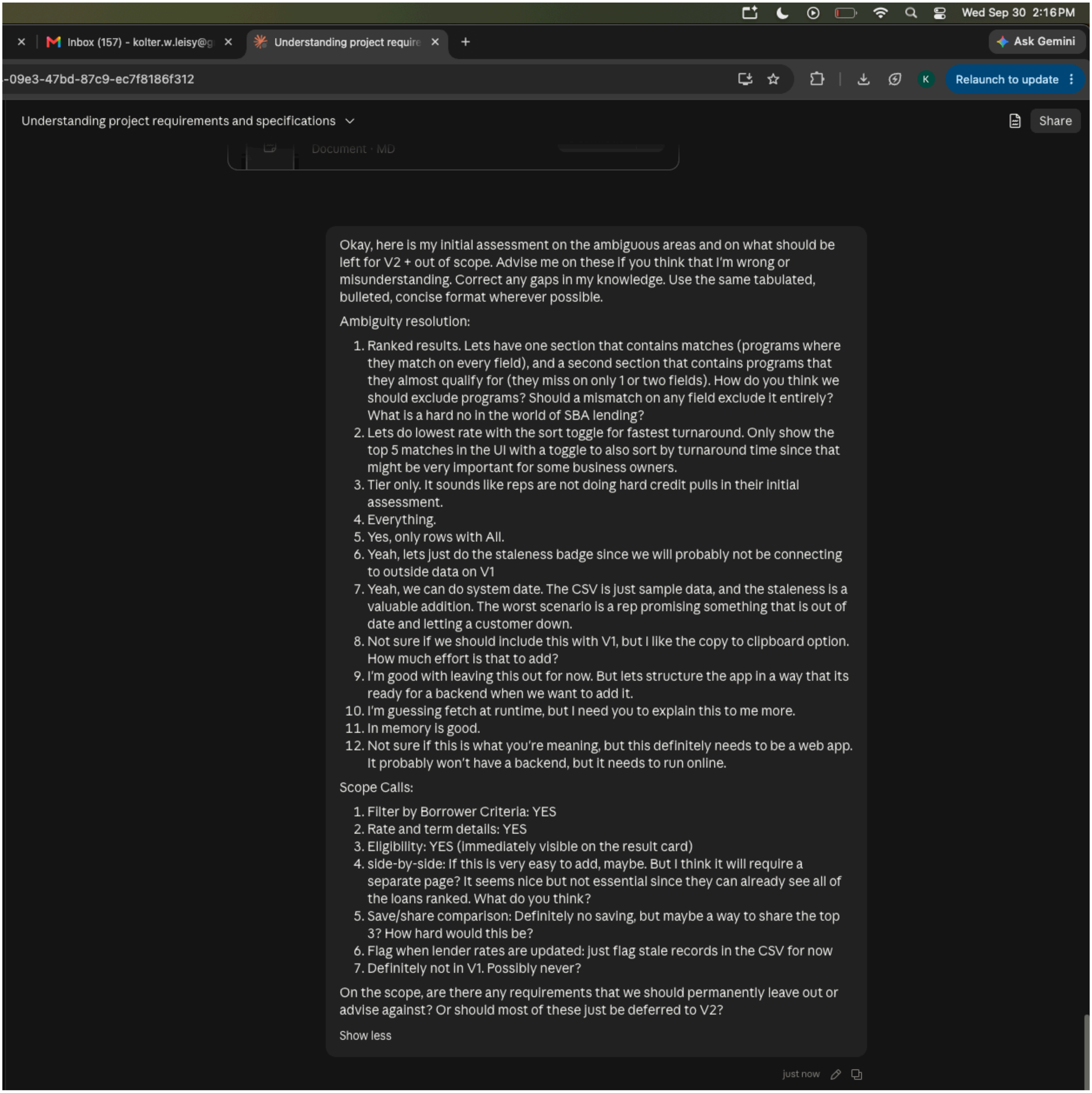
Near-miss handling is the sleeper feature. Real calls aren't clean yes/no. "You're at 18 months; at 24 months these three open up" is the most useful thing the tool can say. Cheap to build, big product-thinking signal.

6. Gaps — decisions you have to make

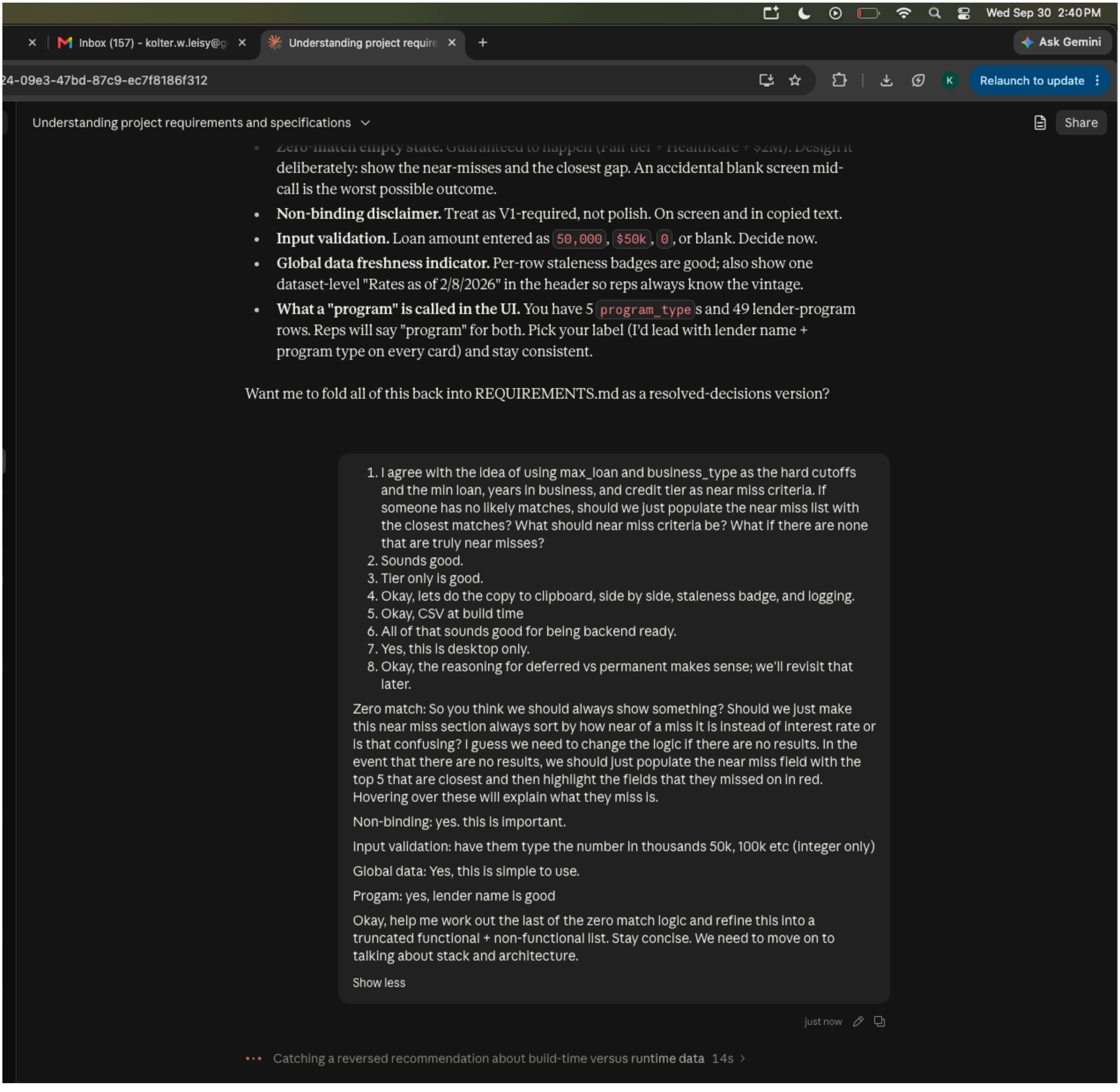
The brief says explicitly: *"make smart calls with incomplete information and explain your reasoning."* These are the calls.

#	Open question	Options	My lean
1	Hard filter or ranked results?	Exclude non-matches / show all with score / show matches + near-misses	Matches first, near-misses in a separate collapsed section with the reason
2	How to rank matches?	Lowest rate / fastest turnaround / highest loan ceiling	Rate ascending, with a sort toggle for turnaround
3	Credit tier vs. credit score input?	Tier dropdown / exact score / both	Tier dropdown (reps have "rough tier"), map tier→score range internally
4	What does "Excellent" match?	Everything / nothing	Everything. Treat tier as a floor
5	Business type not in the list?	"Other" option / free text / omit	"Other" → matches only rows with `All`
6	What does "rates updated" mean?	Changed since last view / older than 30 days / changed vs. previous CSV	Staleness badge if `last_updated` > 30 days before today. Cheapest defensible version
7	"Today" for staleness — real clock or data-relative?	System date / max date in CSV	System date, but note the data is a Feb 2026 snapshot so everything may show stale
8	Share a comparison — how?	Print/PDF / copy-to-clipboard summary / shareable URL	Cut it, or do copy-to-clipboard text if time allows. A URL needs hosting
9	Manager analytics — storage?	localStorage / backend / cut	Cut for V1. 2 users, needs a real backend to be meaningful
10	CSV loading	Bundled at build / fetched at runtime / user uploads	Fetched at runtime so ops can swap the file without a rebuild
11	Is borrower data stored?	Yes / in-memory only	In-memory only. Sidesteps every privacy question
12	Desktop-only or responsive?	—	Desktop-first; reps are at a desk on a headset

03 scope and ambiguity



04 considering no match



For the stack, I'm thinking we can use React. What do you think between TS and JS? We want whichever AI is the best at writing and maintaining.

Identifying which language AI tools handle best overall.

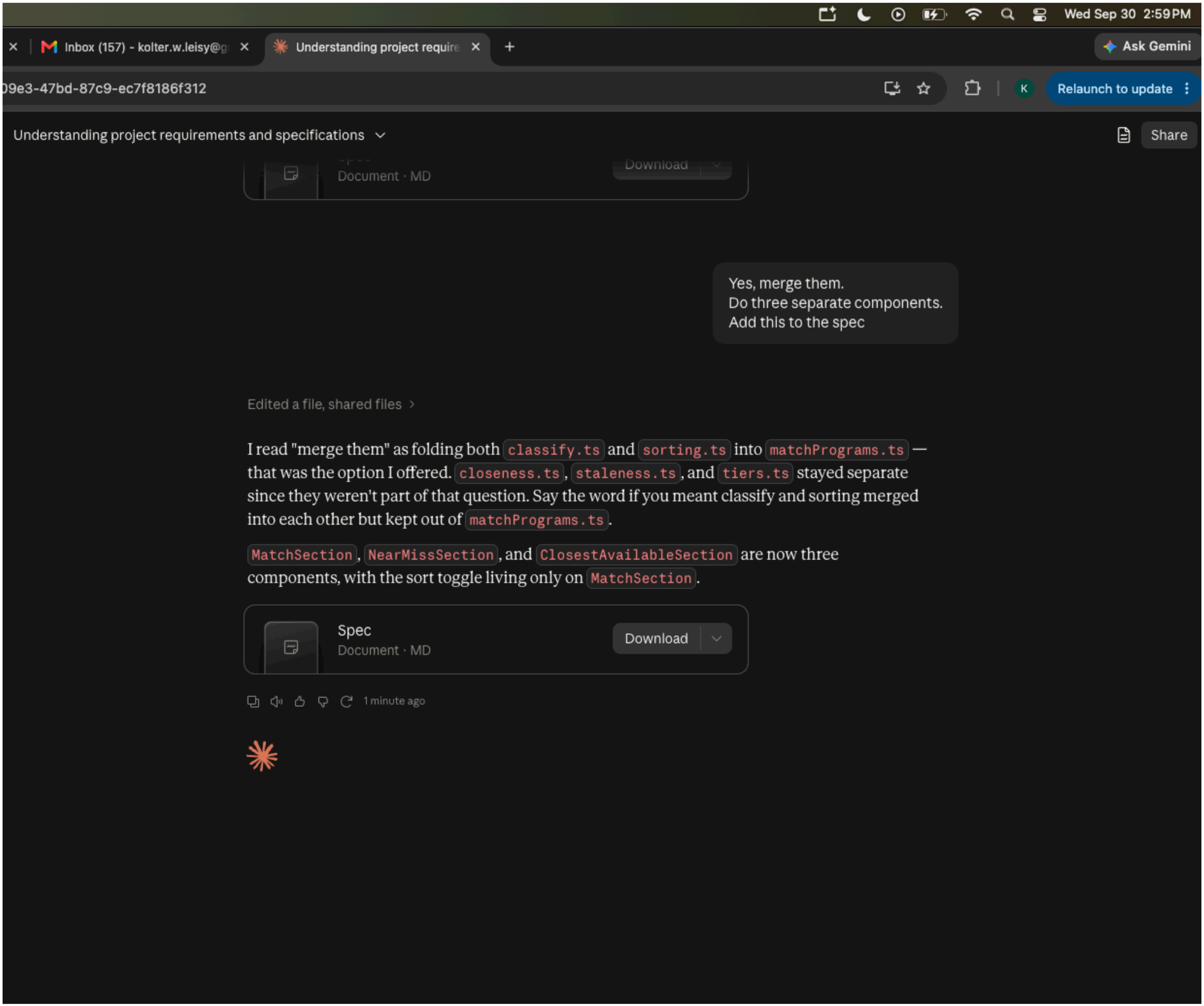
TypeScript. Not close, and the AI angle is actually the strongest argument rather than a wash.

Why TS wins on the "AI writes it best" question

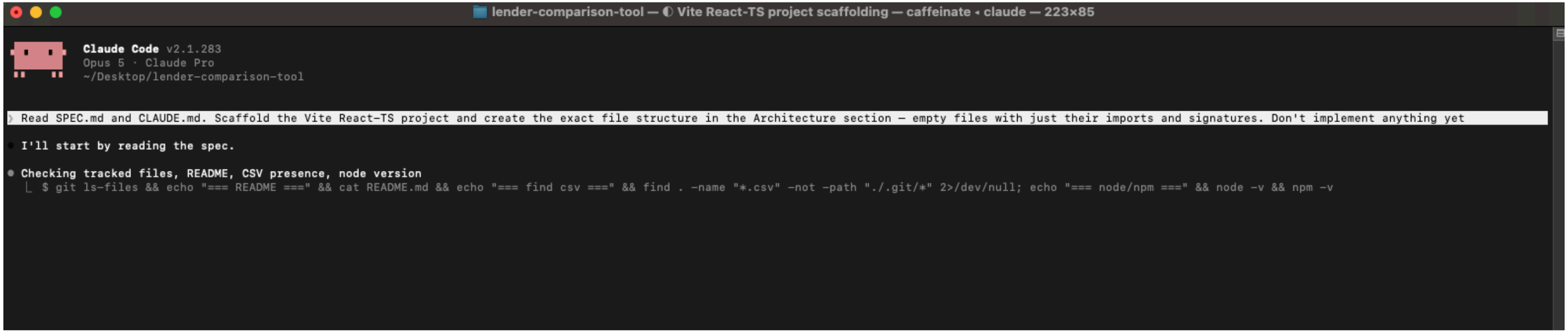
	JavaScript	TypeScript
Raw generation quality	Fine	Fine — both are massively represented in training data
Feedback loop	Errors surface at runtime, in the browser, maybe	Compiler catches them before you run anything
AI self-correction	You have to notice the bug and describe it	Paste the type error, it fixes it
Hallucinated fields	`program.maxLoanAmount` when the CSV says `max_loan_amount` → `undefined`, silently renders blank	Red squiggle, immediately
Refactors ("rename this, change that shape")	AI misses call sites	Compiler enumerates every break

The failure mode that matters here is the third row. You have **17 snake_case CSV columns** that an AI will confidently camelCase half the time. In JS that's a blank cell in a result card that you might not catch before the demo. In TS it doesn't compile.

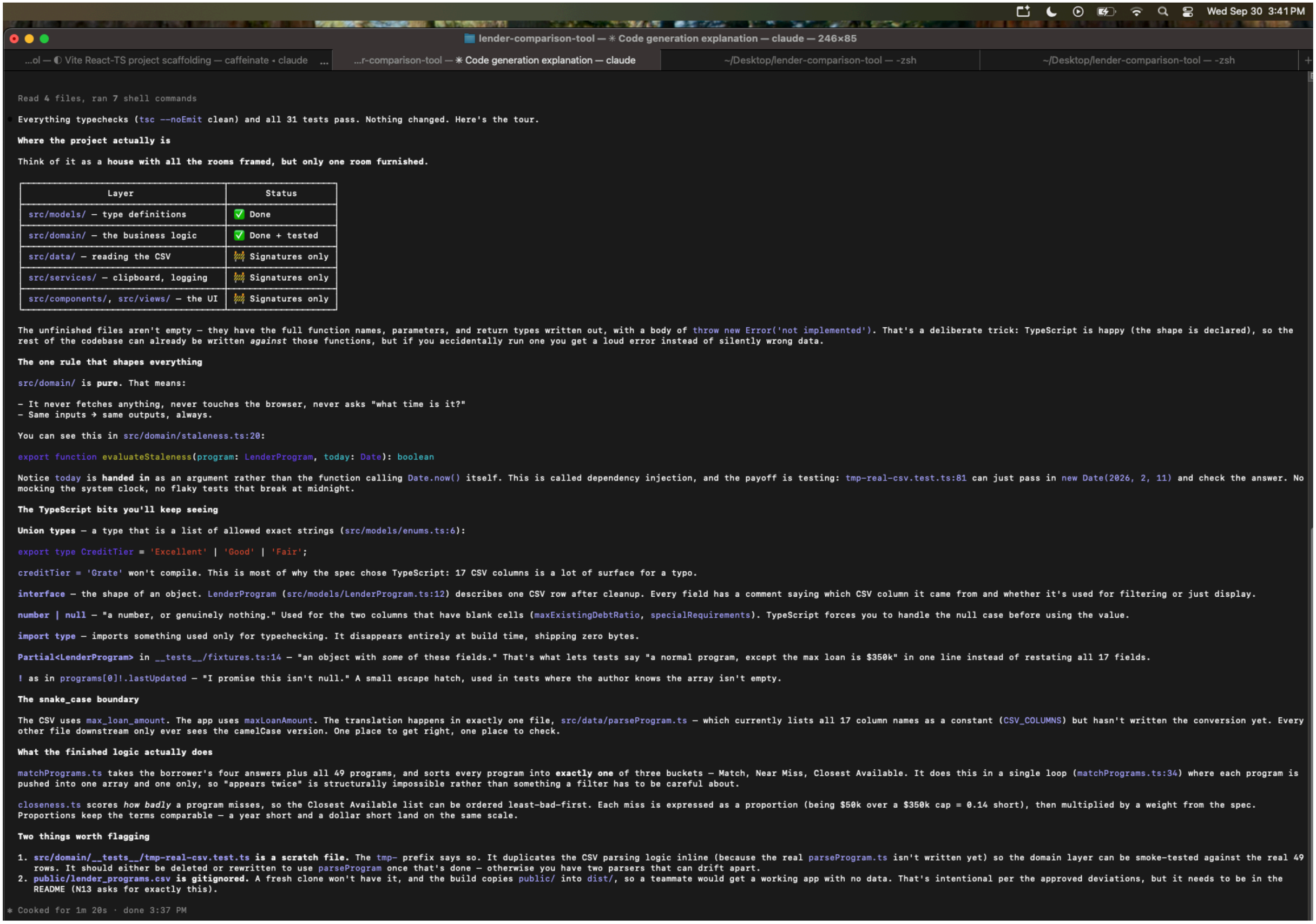
06 final spec file



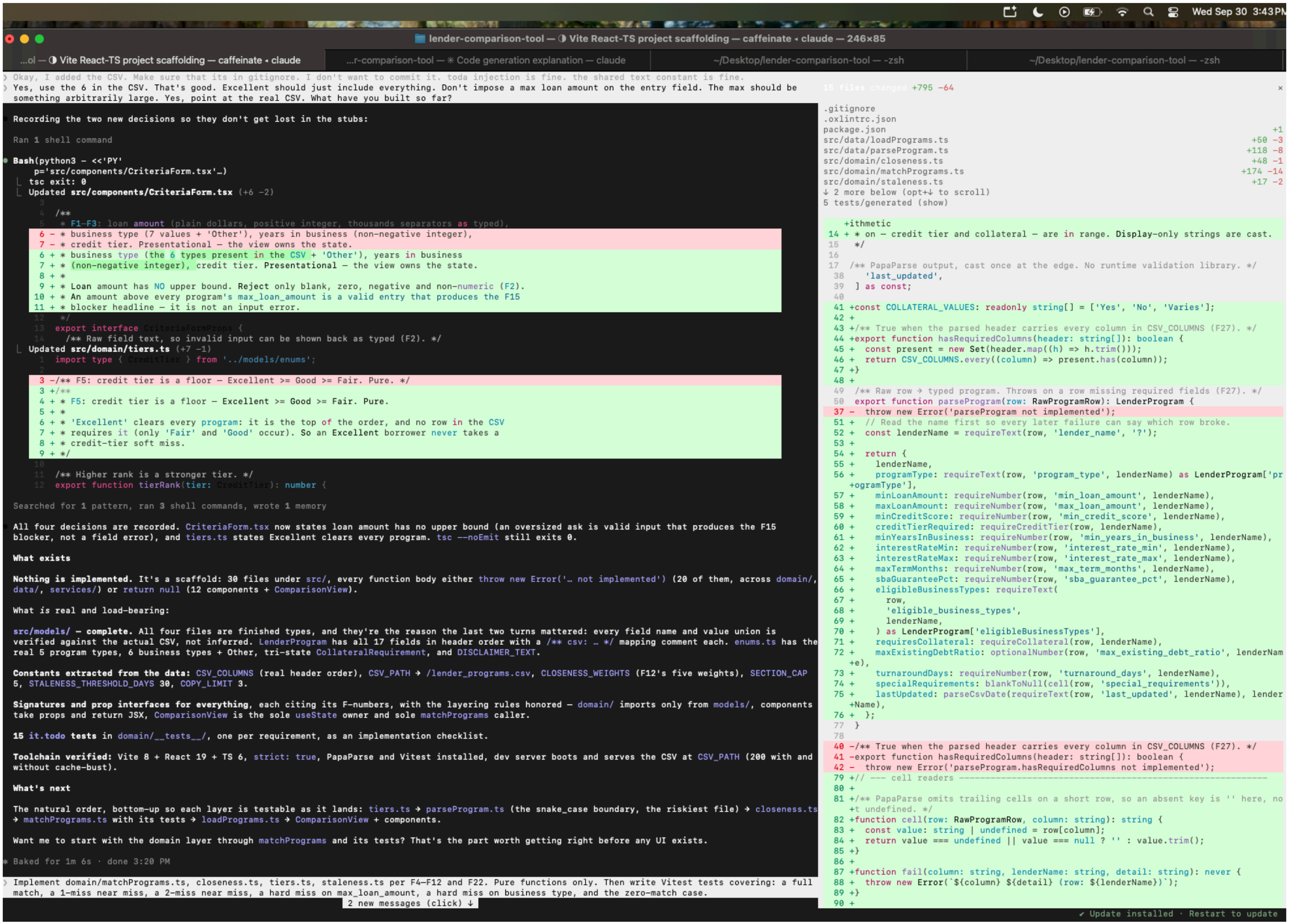
07 build first prompts



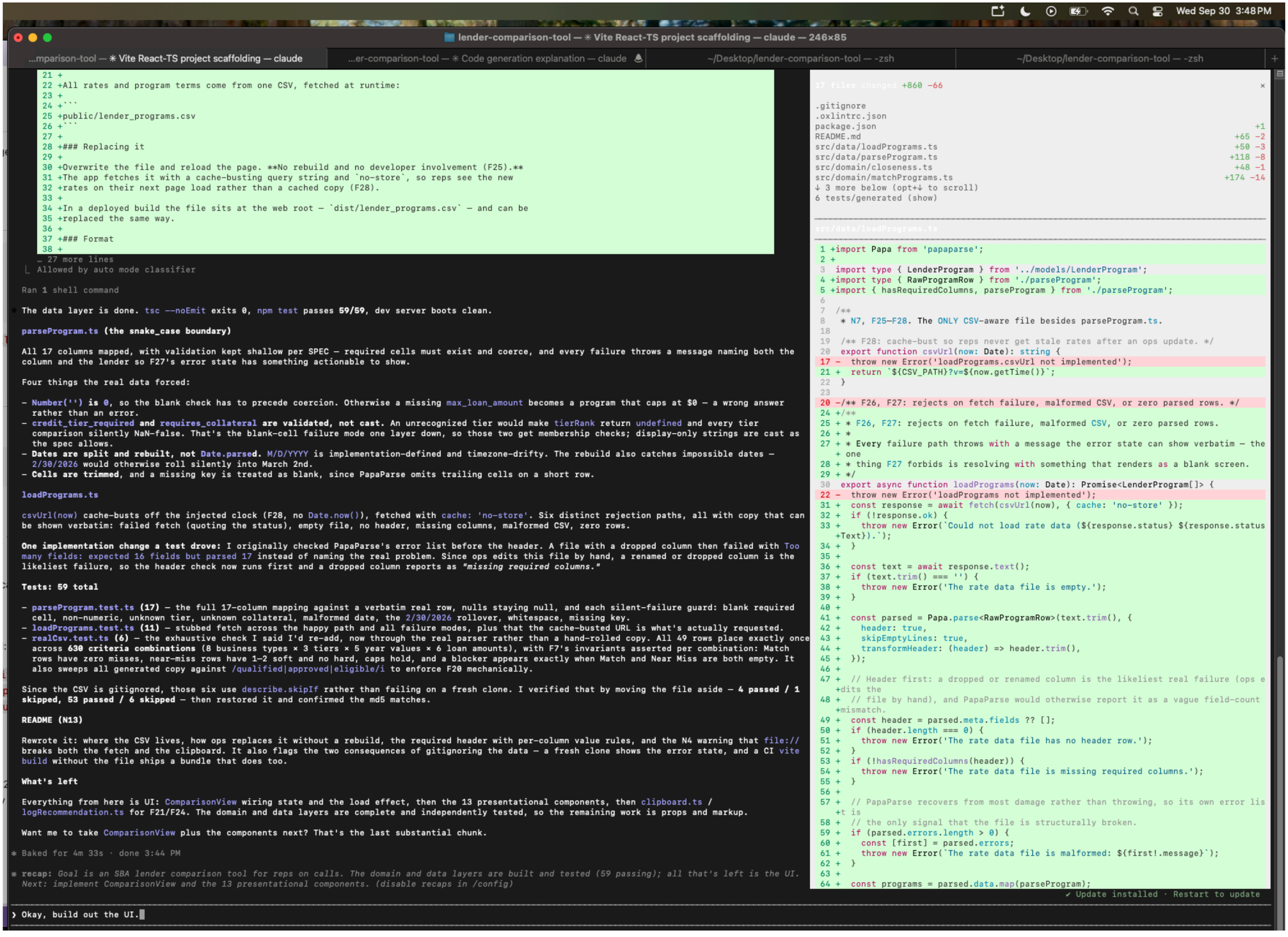
08 reviewing in progress build



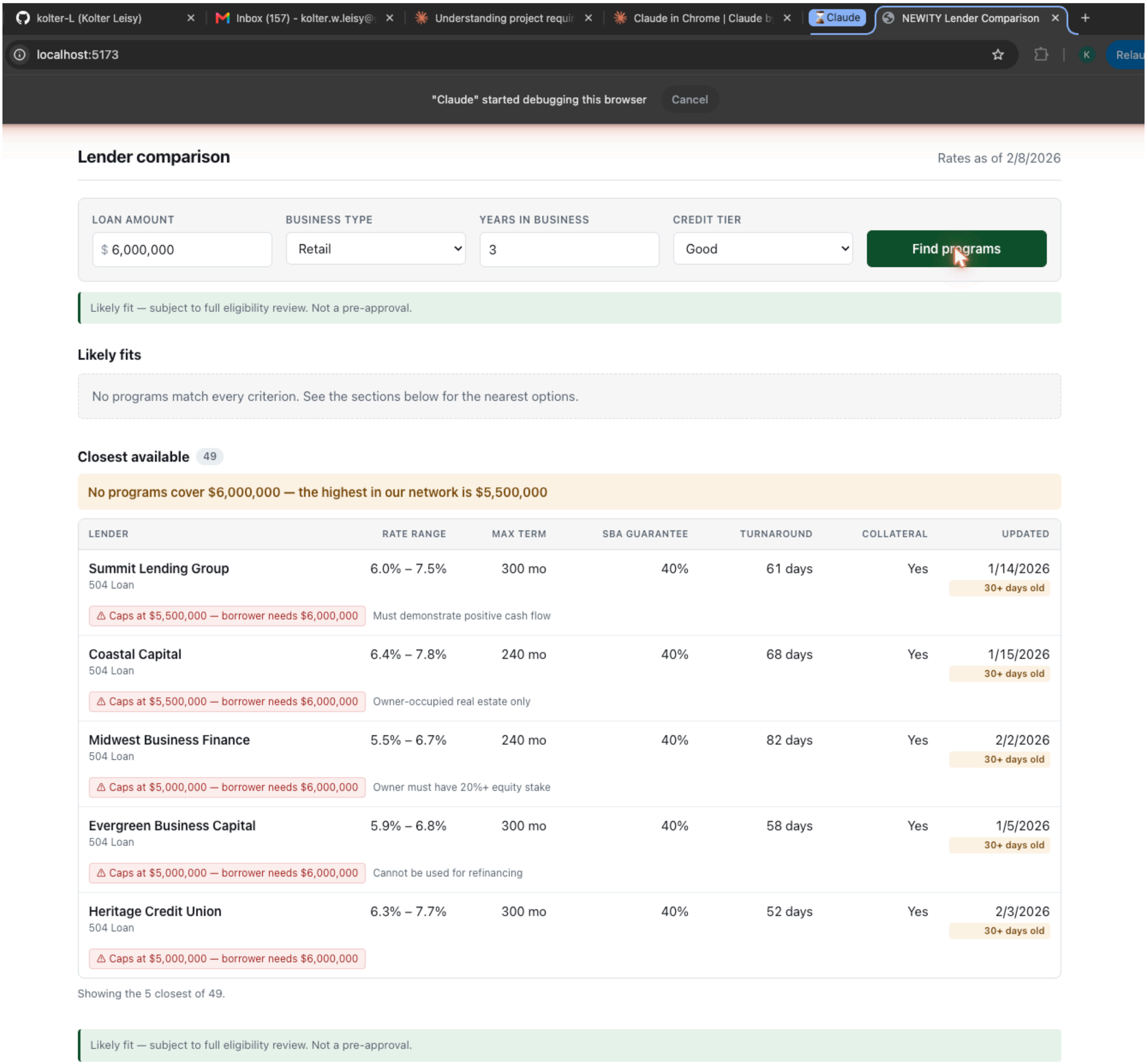
09 implementing matching functions



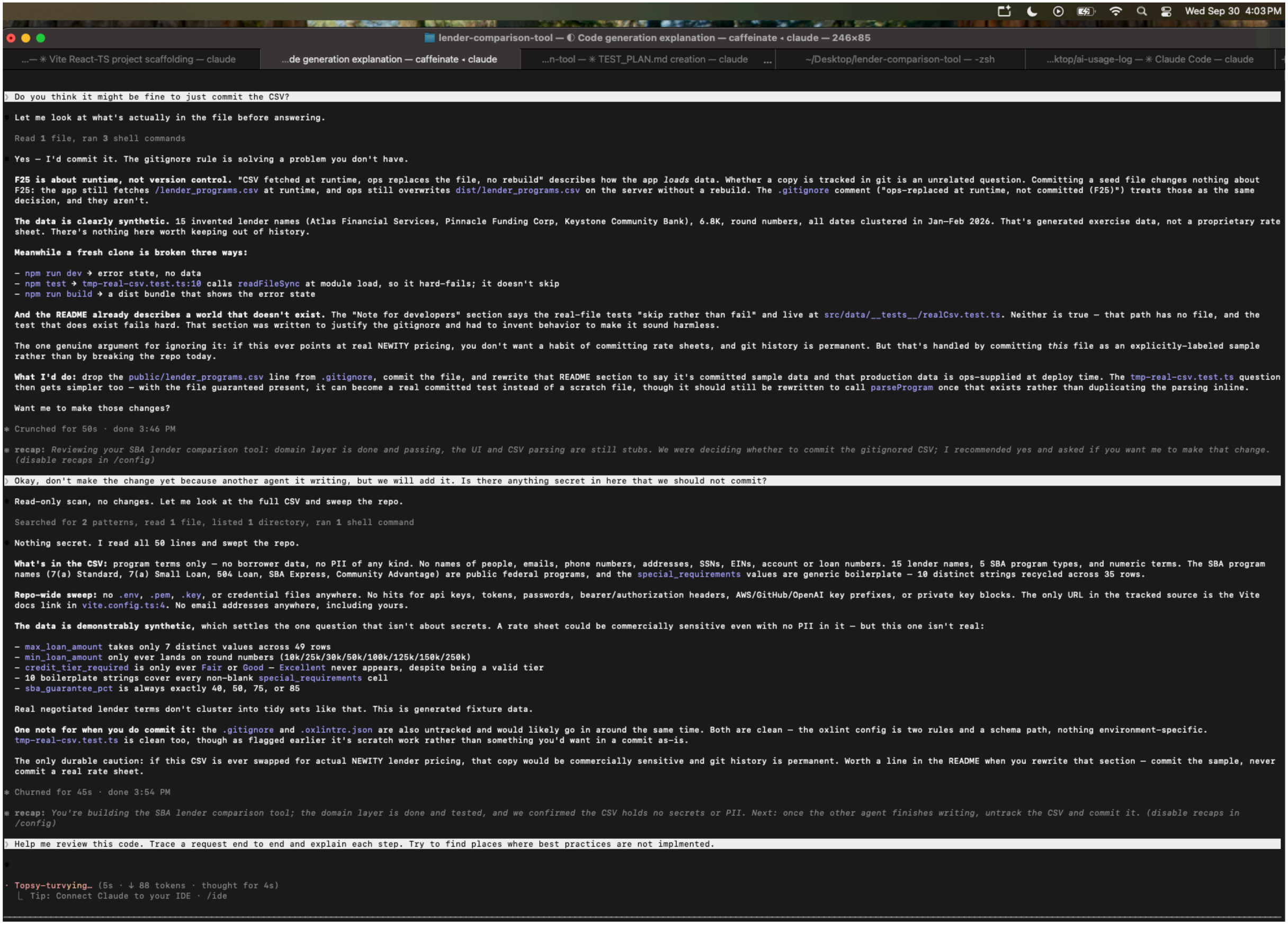
10 build ui


Wed Sep 30 3:48 PM
lender-comparison-tool — ✳ Vite React-TS project scaffolding — claude — 246×85
...mparison-tool — ✳ Vite React-TS project scaffolding — claude
...er-comparison-tool — ✳ Code generation explanation — claude
~/Desktop/lender-comparison-tool — -zsh
~/Desktop/lender-comparison-tool — -zsh
21 +
22 +All rates and program terms come from one CSV, fetched at runtime:
23 +
24 +```
25 +public/lender_programs.csv
26 +```
27 +
28 +### Replacing it
29 +
30 +Overwrite the file and reload the page. **No rebuild and no developer involvement (F25).**
31 +The app fetches it with a cache-busting query string and `no-store`, so reps see the new
32 +rates on their next page load rather than a cached copy (F28).
33 +
34 +In a deployed build the file sits at the web root — `dist/lender_programs.csv` — and can be
35 +replaced the same way.
36 +
37 +### Format
38 +
… 27 more lines
└ Allowed by auto mode classifier
Ran 1 shell command
The data layer is done. tsc --noEmit exits 0, npm test passes 59/59, dev server boots clean.
parseProgram.ts (the snake_case boundary)
All 17 columns mapped, with validation kept shallow per SPEC — required cells must exist and coerce, and every failure throws a message naming both the column and the lender so F27's error state has something actionable to show.
Four things the real data forced:
- Number('') is 0, so the blank check has to precede coercion. Otherwise a missing max_loan_amount becomes a program that caps at $0 — a wrong answer rather than an error.
- credit_tier_required and requires_collateral are validated, not cast. An unrecognized tier would make tierRank return undefined and every tier comparison silently NaN-false. That's the blank-cell failure mode one layer down, so those two get membership checks; display-only strings are cast as the spec allows.
- Dates are split and rebuilt, not Date.parsed. M/D/YYYY is implementation-defined and timezone-drifty. The rebuild also catches impossible dates — 2/30/2026 would otherwise roll silently into March 2nd.
- Cells are trimmed, and a missing key is treated as blank, since PapaParse omits trailing cells on a short row.
loadPrograms.ts
csvUrl(now) cache-busts off the injected clock (F28, no Date.now()), fetched with cache: 'no-store'. Six distinct rejection paths, all with copy that can be shown verbatim: failed fetch (quoting the status), empty file, no header, missing columns, malformed CSV, zero rows.
One implementation change a test drove: I originally checked PapaParse's error list before the header. A file with a dropped column then failed with Too many fields: expected 16 fields but parsed 17 instead of naming the real problem. Since ops edits this file by hand, a renamed or dropped column is the likeliest failure, so the header check now runs first and a dropped column reports as "missing required columns."
Tests: 59 total
- parseProgram.test.ts (17) — the full 17-column mapping against a verbatim real row, nulls staying null, and each silent-failure guard: blank required cell, non-numeric, unknown tier, unknown collateral, malformed date, the 2/30/2026 rollover, whitespace, missing key.
- loadPrograms.test.ts (11) — stubbed fetch across the happy path and all failure modes, plus that the cache-busted URL is what's actually requested.
- realCsv.test.ts (6) — the exhaustive check I said I'd re-add, now through the real parser rather than a hand-rolled copy. All 49 rows place exactly once across 630 criteria combinations (8 business types × 3 tiers × 5 year values × 6 loan amounts), with F7's invariants asserted per combination: Match rows have zero misses, near-miss rows have 1–2 soft and no hard, caps hold, and a blocker appears exactly when Match and Near Miss are both empty. It also sweeps all generated copy against /qualified|approved|eligible/i to enforce F20 mechanically.
Since the CSV is gitignored, those six use describe.skipIf rather than failing on a fresh clone. I verified that by moving the file aside — 4 passed / 1 skipped, 53 passed / 6 skipped — then restored it and confirmed the md5 matches.
README (N13)
Rewrote it: where the CSV lives, how ops replaces it without a rebuild, the required header with per-column value rules, and the N4 warning that file:// breaks both the fetch and the clipboard. It also flags the two consequences of gitignoring the data — a fresh clone shows the error state, and a CI vite build without the file ships a bundle that does too.
What's left
Everything from here is UI: ComparisonView wiring state and the load effect, then the 13 presentational components, then clipboard.ts / logRecommendation.ts for F21/F24. The domain and data layers are complete and independently tested, so the remaining work is props and markup.
Want me to take ComparisonView plus the components next? That's the last substantial chunk.
✻ Baked for 4m 33s · done 3:44 PM
※ recap: Goal is an SBA lender comparison tool for reps on calls. The domain and data layers are built and tested (59 passing); all that's left is the UI. Next: implement ComparisonView and the 13 presentational components. (disable recaps in /config)
17 files changed +860 -66
.gitignore
.oxlintrc.json
package.json +1
README.md +65 -2
src/data/loadPrograms.ts +50 -3
src/data/parseProgram.ts +118 -8
src/domain/closeness.ts +48 -1
src/domain/matchPrograms.ts +174 -14
↓ 3 more below (opt+↓ to scroll)
6 tests/generated (show)
src/data/loadPrograms.ts
1 +import Papa from 'papaparse';
2 +
3 import type { LenderProgram } from '../models/LenderProgram';
4 +import type { RawProgramRow } from './parseProgram';
5 +import { hasRequiredColumns, parseProgram } from './parseProgram';
6
7 /**
8 * N7, F25–F28. The ONLY CSV-aware file besides parseProgram.ts.
18
19 /** F28: cache-bust so reps never get stale rates after an ops update. */
20 export function csvUrl(now: Date): string {
17 - throw new Error('loadPrograms.csvUrl not implemented');
21 + return `${CSV_PATH}?v=${now.getTime()}`;
22 }
23
20 -/** F26, F27: rejects on fetch failure, malformed CSV, or zero parsed rows. */
24 +/**
25 + * F26, F27: rejects on fetch failure, malformed CSV, or zero parsed rows.
26 + *
27 + * Every failure path throws with a message the error state can show verbatim — the
+ one
28 + * thing F27 forbids is resolving with something that renders as a blank screen.
29 + */
30 export async function loadPrograms(now: Date): Promise<LenderProgram[]> {
22 - throw new Error('loadPrograms not implemented');
31 + const response = await fetch(csvUrl(now), { cache: 'no-store' });
32 + if (!response.ok) {
33 + throw new Error(`Could not load rate data (${response.status} ${response.status
+Text}).`);
34 + }
35 +
36 + const text = await response.text();
37 + if (text.trim() === '') {
38 + throw new Error('The rate data file is empty.');
39 + }
40 +
41 + const parsed = Papa.parse<RawProgramRow>(text.trim(), {
42 + header: true,
43 + skipEmptyLines: true,
44 + transformHeader: (header) => header.trim(),
45 + });
46 +
47 + // Header first: a dropped or renamed column is the likeliest real failure (ops e
+dits the
48 + // file by hand), and PapaParse would otherwise report it as a vague field-count
+mismatch.
49 + const header = parsed.meta.fields ?? [];
50 + if (header.length === 0) {
51 + throw new Error('The rate data file has no header row.');
52 + }
53 + if (!hasRequiredColumns(header)) {
54 + throw new Error('The rate data file is missing required columns.');
55 + }
56 +
57 + // PapaParse recovers from most damage rather than throwing, so its own error lis
+t is
58 + // the only signal that the file is structurally broken.
59 + if (parsed.errors.length > 0) {
60 + const [first] = parsed.errors;
61 + throw new Error(`The rate data file is malformed: ${first!.message}`);
62 + }
63 +
64 + const programs = parsed.data.map(parseProgram);
✔ Update installed · Restart to update
❯ Okay, build out the UI.

11 claude testing in chrome



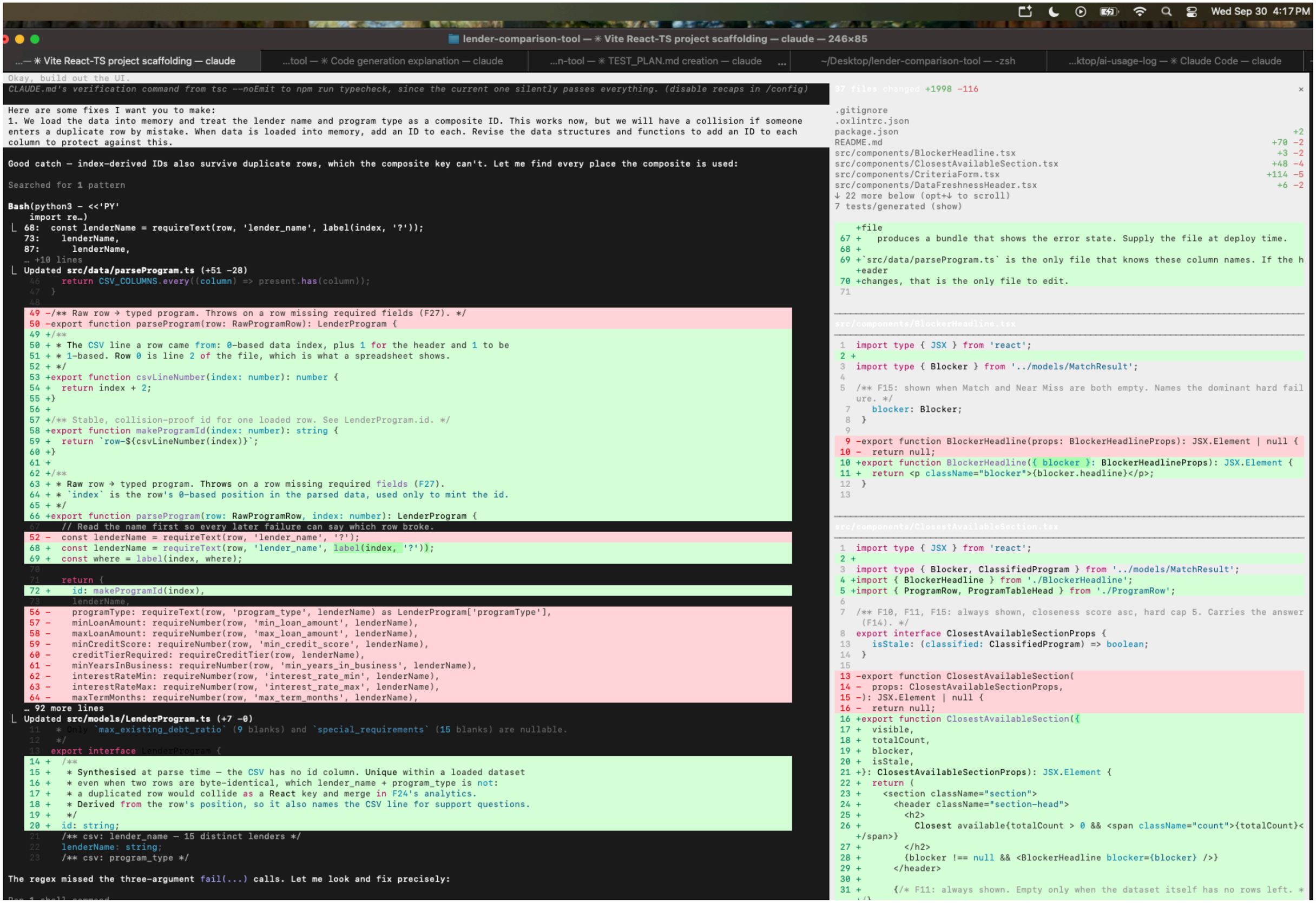
12 starting code review



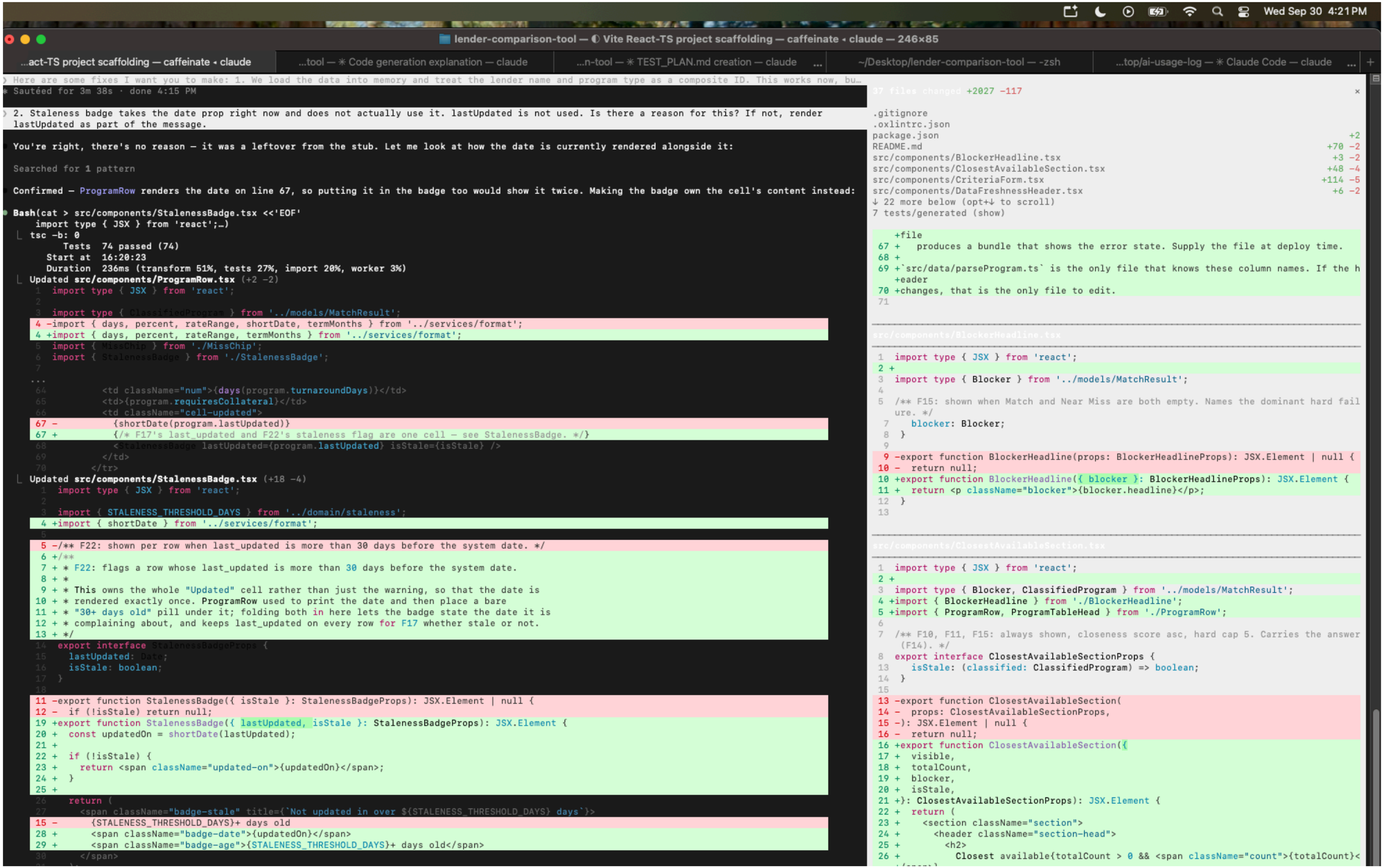
13 initial test plan



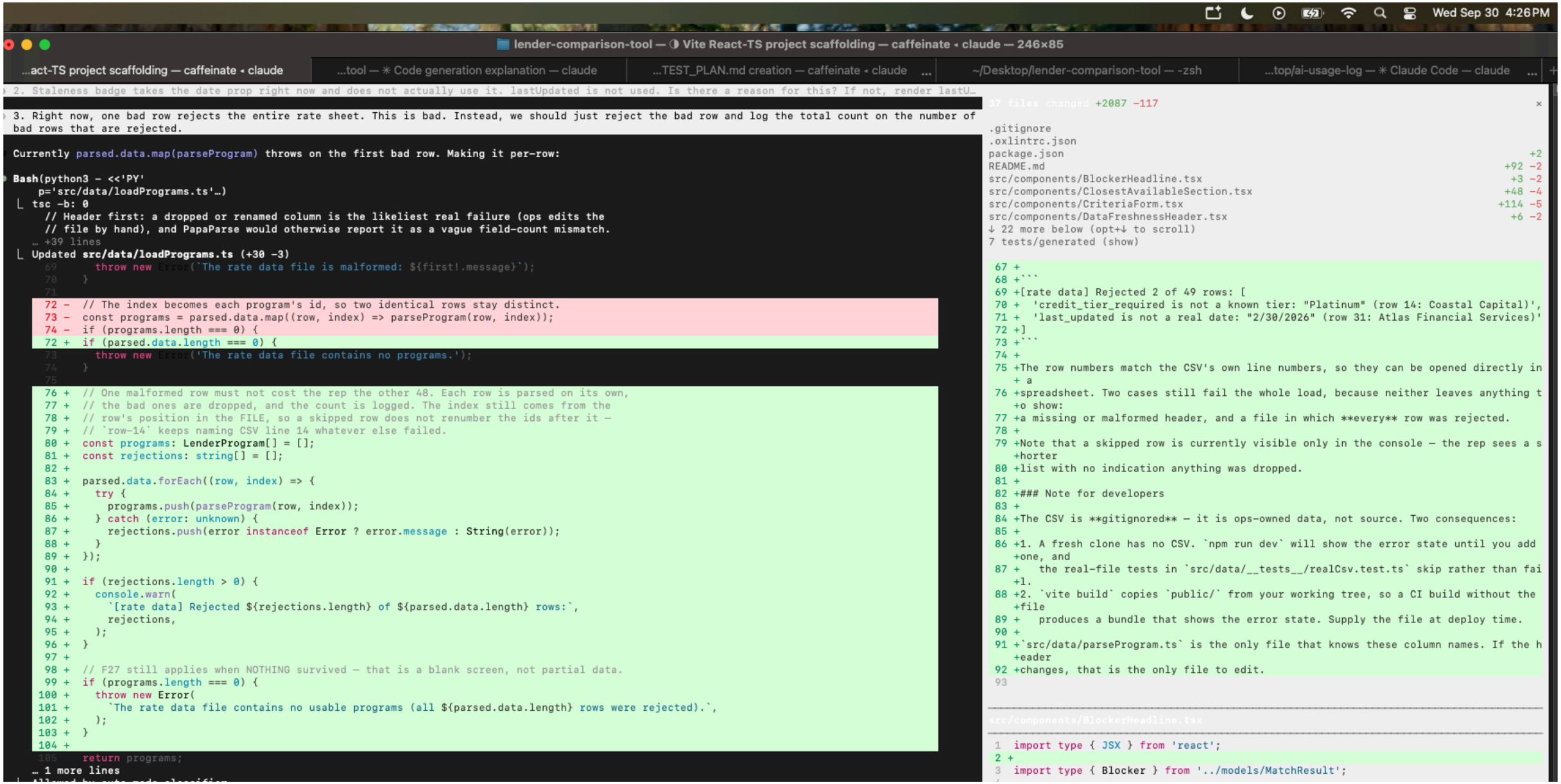
14 adding id column correction



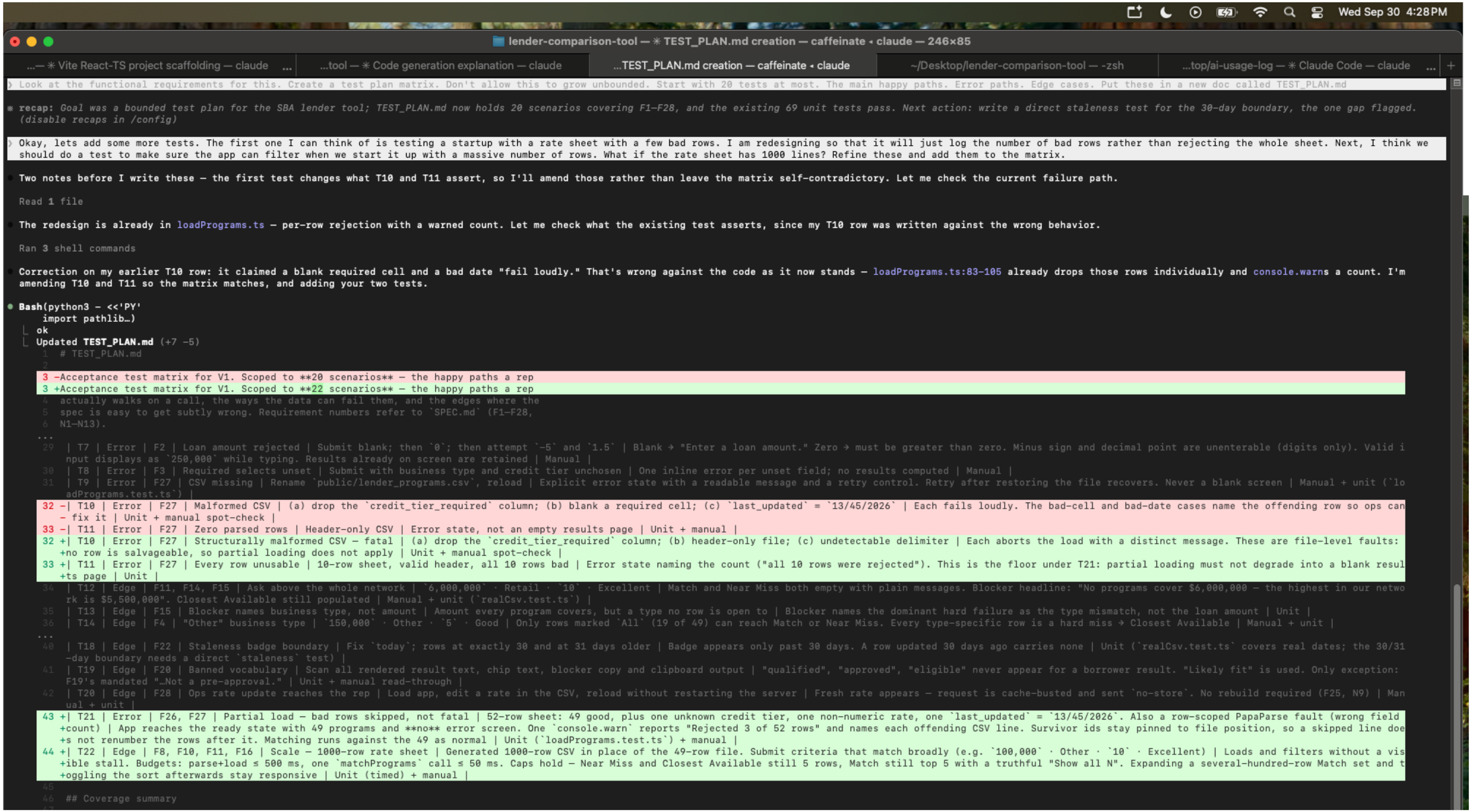
15 fixing unused prop



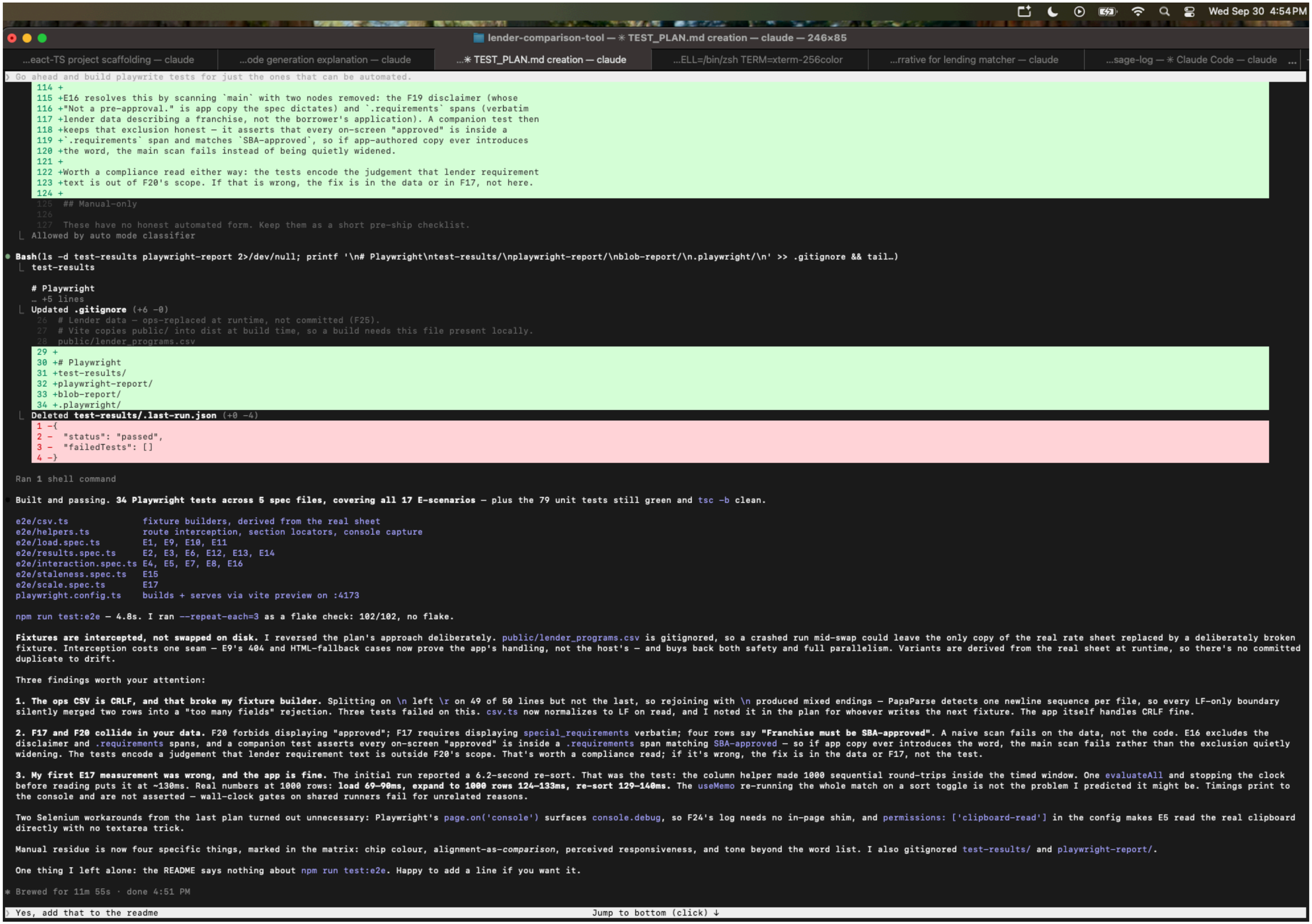
16 log instead of reject data



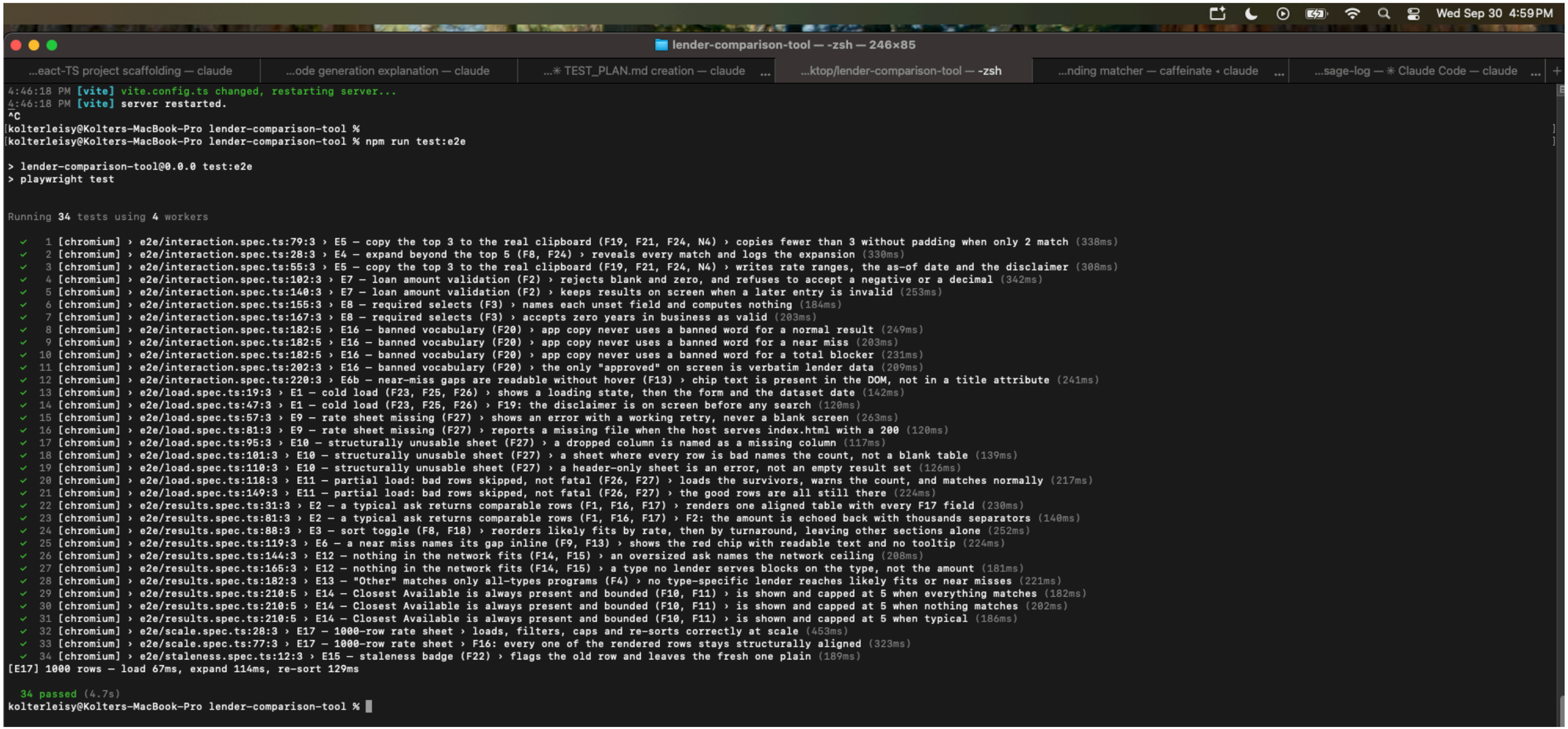
17 update with edge cases



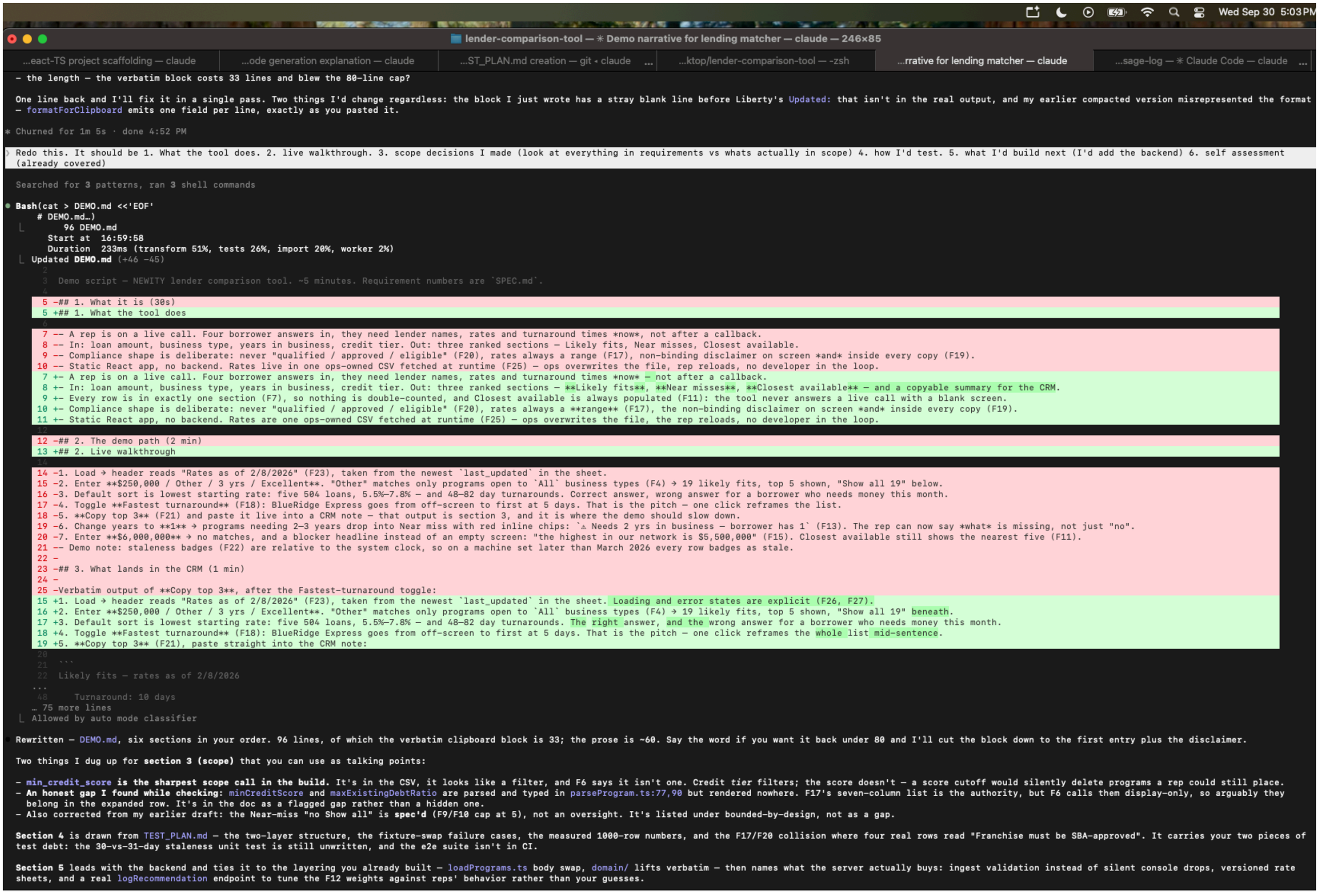
18 e2e tests


Wed Sep 30 4:54 PM
lender-comparison-tool — ✳ TEST_PLAN.md creation — claude — 246×85
...eact-TS project scaffolding — claude
...ode generation explanation — claude
...✳ TEST_PLAN.md creation — claude
...ELL=/bin/zsh TERM=xterm-256color
...rrative for lending matcher — claude
...sage-log — ✳ Claude Code — claude
› Go ahead and build playwrite tests for just the ones that can be automated.
114 +
115 +E16 resolves this by scanning `main` with two nodes removed: the F19 disclaimer (whose
116 +"Not a pre-approval." is app copy the spec dictates) and `.requirements` spans (verbatim
117 +lender data describing a franchise, not the borrower's application). A companion test then
118 +keeps that exclusion honest — it asserts that every on-screen "approved" is inside a
119 +`.requirements` span and matches `SBA-approved`, so if app-authored copy ever introduces
120 +the word, the main scan fails instead of being quietly widened.
121 +
122 +Worth a compliance read either way: the tests encode the judgement that lender requirement
123 +text is out of F20's scope. If that is wrong, the fix is in the data or in F17, not here.
124 +
125 ## Manual-only
126
127 These have no honest automated form. Keep them as a short pre-ship checklist.
Allowed by auto mode classifier
● Bash(ls -d test-results playwright-report 2>/dev/null; printf '\n# Playwright\ntest-results/\nplaywright-report/\nblob-report/\n.playwright/\n' >> .gitignore && tail…)
test-results
Playwright
… +5 lines
Updated .gitignore (+6 −0)
26 # Lender data — ops-replaced at runtime, not committed (F25).
27 # Vite copies public/ into dist at build time, so a build needs this file present locally.
28 public/lender_programs.csv
29 +
30 +# Playwright
31 +test-results/
32 +playwright-report/
33 +blob-report/
34 +.playwright/
Deleted test-results/.last-run.json (+0 −4)
1 -{
2 - "status": "passed",
3 - "failedTests": []
4 -}
Ran 1 shell command
● Built and passing. 34 Playwright tests across 5 spec files, covering all 17 E-scenarios — plus the 79 unit tests still green and tsc -b clean.
e2e/csv.ts fixture builders, derived from the real sheet
e2e/helpers.ts route interception, section locators, console capture
e2e/load.spec.ts E1, E9, E10, E11
e2e/results.spec.ts E2, E3, E6, E12, E13, E14
e2e/interaction.spec.ts E4, E5, E7, E8, E16
e2e/staleness.spec.ts E15
e2e/scale.spec.ts E17
playwright.config.ts builds + serves via vite preview on :4173
npm run test:e2e — 4.8s. I ran --repeat-each=3 as a flake check: 102/102, no flake.
Fixtures are intercepted, not swapped on disk. I reversed the plan's approach deliberately. public/lender_programs.csv is gitignored, so a crashed run mid-swap could leave the only copy of the real rate sheet replaced by a deliberately broken fixture. Interception costs one seam — E9's 404 and HTML-fallback cases now prove the app's handling, not the host's — and buys back both safety and full parallelism. Variants are derived from the real sheet at runtime, so there's no committed duplicate to drift.
Three findings worth your attention:
1. The ops CSV is CRLF, and that broke my fixture builder. Splitting on \n left \r on 49 of 50 lines but not the last, so rejoining with \n produced mixed endings — PapaParse detects one newline sequence per file, so every LF-only boundary silently merged two rows into a "too many fields" rejection. Three tests failed on this. csv.ts now normalizes to LF on read, and I noted it in the plan for whoever writes the next fixture. The app itself handles CRLF fine.
2. F17 and F20 collide in your data. F20 forbids displaying "approved"; F17 requires displaying special_requirements verbatim; four rows say "Franchise must be SBA-approved". A naive scan fails on the data, not the code. E16 excludes the disclaimer and .requirements spans, and a companion test asserts every on-screen "approved" is inside a .requirements span matching SBA-approved — so if app copy ever introduces the word, the main scan fails rather than the exclusion quietly widening. The tests encode a judgement that lender requirement text is outside F20's scope. That's worth a compliance read; if it's wrong, the fix is in the data or F17, not the test.
3. My first E17 measurement was wrong, and the app is fine. The initial run reported a 6.2-second re-sort. That was the test: the column helper made 1000 sequential round-trips inside the timed window. One evaluateAll and stopping the clock before reading puts it at ~130ms. Real numbers at 1000 rows: load 69–90ms, expand to 1000 rows 124–133ms, re-sort 129–140ms. The useMemo re-running the whole match on a sort toggle is not the problem I predicted it might be. Timings print to the console and are not asserted — wall-clock gates on shared runners fail for unrelated reasons.
Two Selenium workarounds from the last plan turned out unnecessary: Playwright's page.on('console') surfaces console.debug, so F24's log needs no in-page shim, and permissions: ['clipboard-read'] in the config makes E5 read the real clipboard directly with no textarea trick.
Manual residue is now four specific things, marked in the matrix: chip colour, alignment-as-comparison, perceived responsiveness, and tone beyond the word list. I also gitignored test-results/ and playwright-report/.
One thing I left alone: the README says nothing about npm run test:e2e. Happy to add a line if you want it.
✻ Brewed for 11m 55s · done 4:51 PM
› Yes, add that to the readme
Jump to bottom (click) ↓

19 passing e2e tests



20 draft demo script



21 update readme

